

CMIA

Whitepaper

Une blockchain Proof of Work à minage ouvert

Version	1.0
Date	Octobre 2026
Statut	Expérimental
Réseau	cmia-mainnet-v1
Site	cryptomonnaiesia.com
Licence du logiciel	MIT

Sommaire

1. Résumé	3
2. Principes	3
3. Architecture	3
4. Adresses, transactions et signatures	3
5. Blocs	4
6. Consensus	4
7. Minage ouvert	5
8. Réseau	6
9. Politique monétaire	6
10. Sécurité et modèle de menace	7
11. Limites connues et pistes d'évolution	7
12. Avertissement	8
Annexe A. Paramètres du réseau	8
Annexe B. Interfaces principales	8
Annexe C. Ressources	9

1. Résumé

CMIA est une blockchain publique en **Proof of Work** (SHA-256). Elle repose sur le modèle **UTXO** et sur des signatures **secp256k1** (ECDSA). Le minage est **ouvert** : n'importe qui peut miner, sans inscription ni autorisation.

Le nœud fournit un modèle de bloc ; le mineur calcule le Proof of Work **sur sa propre machine** ; le nœud vérifie toutes les règles de consensus avant d'accepter le bloc. Le mineur ne transmet jamais sa clé privée : seule son adresse publique reçoit la récompense.

L'émission totale est plafonnée à **10 000 000 CMIA**. Elle ne provient que de récompenses de bloc : le bloc Genesis ne crée aucune unité. Le nœud et le mineur sont publiés sous licence MIT.

CMIA est un projet expérimental. Ce document décrit ce que le code fait aujourd'hui, ses limites connues et les pistes d'amélioration. Il ne promet aucune valeur ni aucun rendement.

2. Principes

- **Minage sans permission.** Aucune liste de mineurs autorisés, aucun secret à demander à un opérateur.
- **Clés chez leurs propriétaires.** Les transactions et le portefeuille du mineur sont signés localement.
- **Règles lisibles.** Le consensus tient dans un seul fichier Python (`core/blockchain.py`), que chacun peut relire et vérifier.
- **Simple à faire tourner.** Un nœud est une application Python avec une base SQLite : quelques commandes suffisent.
- **En français.** Documentation et communauté francophones.

3. Architecture

Composant	Rôle
Nœud	Valide les blocs et les transactions, stocke la chaîne, expose l'API REST (FastAPI).
Mineur	Client Python : récupère un modèle de bloc, cherche le nonce, soumet le bloc.
Portefeuille	Outil en ligne de commande : adresse, solde, envoi signé localement.
Réseau pair-à-pair	Diffusion TCP (JSON, une ligne par message) des blocs et des transactions.
Synchronisation	Un nœud rattrape son retard par HTTP, lots de 100 blocs, en validant chaque bloc.
Stockage	SQLite par défaut, PostgreSQL possible. Les sorties non dépensées (UTXO) sont indexées.

4. Adresses, transactions et signatures

4.1 Adresses et montants

Une adresse est une clé publique secp256k1 non compressée : 64 octets, soit 128 caractères hexadécimaux. Les montants sont des entiers ; la plus petite unité est 1 CMIA.

4.2 Transactions

Une transaction consomme des sorties non dépensées (entrées : `prev_tx_hash` et `output_index`) et crée de nouvelles sorties (`recipient_pub` et `amount`). Toutes les entrées doivent appartenir à l'expéditeur. La différence entre entrées et sorties constitue des frais implicites.

L'expéditeur signe, en ECDSA sur SHA-256, un **message d'intention** : un JSON canonique contenant l'identifiant du réseau, la version de transaction, l'expéditeur, les entrées (sans leur signature), les sorties et les frais. Intégrer l'identifiant du réseau et la version empêche le jeu d'une transaction sur une autre chaîne.

Le `tx_hash` est le SHA-256 du JSON canonique (expéditeur, entrées, sorties, signature, date de création). Chaque nœud le recalcule : une transaction dont le hash ne correspond pas est rejetée.

4.3 Transaction coinbase

Chaque bloc commence par une transaction **coinbase** : l'expéditeur est `NETWORK`, le type est `coinbase`, le montant est la récompense du bloc, le destinataire est l'adresse du mineur. Il y en a exactement une par bloc, toujours en première position.

Les frais de transaction ne sont pas reversés aux mineurs : ils sont détruits. Un mécanisme de frais pour les mineurs est une piste d'évolution.

5. Blocs

Un bloc contient exactement sept champs : `height`, `prev_hash`, `timestamp`, `difficulty`, `transactions`, `nonce` et `hash`. Le hash est le SHA-256 du JSON canonique du bloc (clés triées, séparateurs compacts) **sans** le champ `hash`. Tout champ supplémentaire invalide le bloc, car il modifierait le hash chez les autres nœuds.

Le **bloc Genesis** (hauteur 0) a pour horodatage 1 760 000 000 (9 octobre 2025, 08:53:20 UTC), une difficulté de 3 et une unique transaction de type `genesis` de montant 0, qui porte un message d'identification du réseau.

6. Consensus

6.1 Proof of Work

La difficulté d est exprimée en **bits de travail**. Un bloc est valide si son hash, lu comme un entier de 256 bits, est inférieur à 2^{256-d} . Il faut donc en moyenne environ 2^d essais pour en trouver un : chaque bit de difficulté double le travail.

6.2 Ajustement de la difficulté

La difficulté est recalculée tous les **5 blocs** pour viser un bloc toutes les **30 secondes**. Le réseau compare la durée réelle des 5 derniers intervalles à la durée attendue (150 secondes) :

- le rapport durée réelle / durée attendue est borné entre 0,25 et 4 ;

- si les blocs arrivent trop vite, la difficulté augmente d'au moins 1 bit ; s'ils arrivent trop lentement, elle diminue d'au moins 1 bit ;
- la variation vaut l'arrondi de $\log_2$ du rapport, soit 2 bits au plus par ajustement ;
- la difficulté reste comprise entre 3 et 240 bits.

La difficulté d'un bloc n'est pas choisie par le mineur : elle est **imposée** et chaque nœud la recalcule.

6.3 Règles de validité d'un bloc

Règle	Détail
Forme	Exactement les sept champs ; au plus 2 Mo ; hash de 64 caractères hexadécimaux.
Hash	Le hash annoncé est celui que le nœud recalcule.
Proof of Work	Le hash est inférieur à la cible de la difficulté.
Chaînage	La hauteur est celle de la pointe plus 1 ; <code>prev_hash</code> est le hash de la pointe.
Difficulté	Égale à la difficulté de consensus calculée par le nœud.
Horodatage	Au plus 60 secondes dans le futur ; au plus 60 secondes avant le bloc précédent.
Coinbase	Une seule, en première position, au montant exact de la récompense, avec un <code>tx_hash</code> valide.
Plafond	La supply après le bloc ne dépasse jamais 10 000 000 CMIA.
Transactions	200 au plus ; chaque <code>tx_hash</code> et chaque signature vérifiés ; sorties existantes, non dépensées et possédées par l'expéditeur ; aucune double dépense dans le bloc.

6.4 Choix de la chaîne

Dans la version actuelle, un nœud accepte un bloc uniquement s'il prolonge sa pointe. Il n'existe pas encore de règle de réorganisation vers la chaîne au plus grand travail cumulé : voir la section 11.

7. Minage ouvert

7.1 Protocole

Mineur	Nœud
GET /mining/template?miner_pub=...	
----->	modele de bloc : hauteur, prev_hash,
<-----	difficulté, coinbase, transactions
recherche du nonce (SHA-256)	
GET /mining/tip (toutes les 3 s)	abandon si la pointe a change
POST /mining/submit (bloc complet)	
----->	validation complete du consensus
<-----	200 accepte 409 depasse 400 refuse

7.2 Propriétés

- **Sans inscription.** Seule une adresse publique est nécessaire ; aucun identifiant ni secret n'est échangé.
- **Calcul chez le mineur.** Le nœud ne mine jamais à la place d'un client : il vérifie.
- **Confidentialité de la clé.** La clé privée reste dans le portefeuille du mineur.

- **Concurrence.** Le premier bloc valide à une hauteur donnée est retenu ; les mineurs plus lents reçoivent une réponse 409 et repartent sur le bloc suivant.
- **Modèle sûr.** Le nœud n'inclut dans le modèle que des transactions qu'il a lui-même validées : une transaction douteuse ne peut pas invalider le bloc.
- **Transparence.** `GET /miners` publie les statistiques et le classement des mineurs.

8. Réseau

- **API HTTP.** Lecture publique de la chaîne, de la mempool, des soldes et des sorties non dépensées ; réception des blocs, des transactions et du minage.
- **P2P TCP.** Les nœuds se diffusent les nouveaux blocs et transactions ; chaque message reçu est revalidé complètement par le nœud destinataire avant acceptation.
- **Synchronisation initiale.** Un nœud récupère les blocs manquants par lots de 100 depuis un nœud choisi et vérifie chacun avec ses propres règles. Il refuse de suivre un pair dont le Genesis est différent.
- **Mempool.** 5 000 transactions au plus, 50 par expéditeur, 25 000 octets par transaction.
- **Anti-abus.** Limitation du débit par adresse IP ; routes d'administration fermées tant qu'aucun secret n'est configuré.

9. Politique monétaire

9.1 Principes

- **Plafond :** 10 000 000 CMIA.
- **Aucune préminée dans le Genesis** (montant 0). Par construction, la supply est la somme des récompenses de bloc : une transaction ne peut que déplacer des unités existantes.
- **Récompense :** 25 CMIA au départ, divisée par deux (division entière, minimum 1) tous les 5 000 blocs.
- **Frais :** détruits (voir 4.3).

9.2 Calendrier d'émission

Période	Hauteurs	Récompense	Émis dans la période	Total émis
1	1 - 4 999	25	124 975	124 975
2	5 000 - 9 999	12	60 000	184 975
3	10 000 - 14 999	6	30 000	214 975
4	15 000 - 19 999	3	15 000	229 975
5 et suivantes	à partir de 20 000	1	1 par bloc	jusqu'à 10 000 000

À la cadence cible de 30 secondes par bloc, les quatre premières périodes (20 000 blocs) durent environ **7 jours** et émettent **2,3 %** du plafond. Le reste est émis à 1 CMIA par bloc : le plafond est atteint au bloc 9 790 024, soit environ **9,3 ans** au total.

Ces durées sont théoriques : la cadence réelle dépend de la puissance de calcul du réseau, et la difficulté s'ajuste en conséquence. Le calendrier d'émission est un paramètre de consensus : le

modifier crée une chaîne incompatible.

10. Sécurité et modèle de menace

Menace	Situation actuelle
Bloc falsifié (difficulté, récompense, coinbase multiple, tx_hash, champs en trop)	Rejeté par la validation de consensus, que chaque nœud applique.
Double dépense	Une sortie dépensée est marquée comme telle ; une double dépense dans un même bloc est détectée ; la mempool réserve les entrées.
Rejeu d'une transaction sur un autre réseau	L'identifiant du réseau et la version sont inclus dans le message signé.
Attaque à 51 %	Sur un réseau jeune, un mineur majoritaire peut réécrire les derniers blocs ou censurer des transactions. Aucune protection de type point de contrôle.
Manipulation des horodatages	Une dérive de plus ou moins 60 secondes est tolérée ; combinée à une fenêtre d'ajustement de 5 blocs, elle peut influencer la difficulté.
Divergence de branches	Pas de réorganisation : deux nœuds qui acceptent des blocs différents à la même hauteur peuvent rester séparés.
Spam et surcharge	Limites de mempool, de taille et de débit par IP. Le port P2P n'est pas authentifié.
Vol de clés	Mineurs et utilisateurs gardent leurs clés. Les wallets gérés par un nœud (routes d'administration) stockent leurs clés en clair tant que Vault n'est pas activé : à ne pas utiliser sur un nœud public.

11. Limites connues et pistes d'évolution

11.1 Limites de la version 1

- Pas de réorganisation de chaîne ni de calcul du travail cumulé.
- Fenêtre d'ajustement de difficulté courte (5 blocs) et granularité d'un bit, donc des variations d'un facteur 2.
- Réseau P2P sans authentification ni protection contre les envois massifs.
- Frais de transaction non reversés aux mineurs.
- Mineur écrit en Python : performances modestes.
- Code non audité par un tiers.

11.2 Pistes d'évolution

- Travail cumulé, détection de fork et réorganisation vers la chaîne la plus lourde.
- Fenêtre de difficulté plus longue et ajustement plus fin.
- Poignée de main et authentification entre pairs.
- Frais reversés aux mineurs.
- Explorateur de blocs et portefeuille graphique.
- Tests automatisés du consensus et audit externe.

Aucune date n'est promise : l'ordre dépend des retours de la communauté.

12. Avertissement

CMIA est un logiciel expérimental fourni « tel quel », sans garantie d'aucune sorte. Ce document est une description technique ; il ne constitue ni un conseil en investissement, ni une offre, ni une promesse de valeur, de liquidité ou de rendement. Les unités CMIA peuvent n'avoir aucune valeur et leur perte est possible, notamment en cas de perte de clé privée ou de bug. Chacun reste responsable du respect de la réglementation applicable dans son pays.

Annexe A. Paramètres du réseau

Paramètre	Valeur
Identifiant du réseau	cmia-mainnet-v1
Version de transaction	1
Hachage	SHA-256
Signature	ECDSA, courbe secp256k1
Genesis	hauteur 0, horodatage 1 760 000 000, difficulté 3
Difficulté minimale / maximale	3 / 240 bits
Fenêtre d'ajustement	5 blocs
Temps de bloc cible	30 secondes
Tolérance d'horodatage	+60 s dans le futur ; -60 s avant le bloc précédent
Récompense initiale	25 CMIA
Période de division par deux	5 000 blocs ; récompense minimale 1 CMIA
Plafond de supply	10 000 000 CMIA
Taille maximale d'un bloc	2 000 000 octets ; 200 transactions
Mempool	5 000 transactions ; 50 par expéditeur ; 25 000 octets par transaction
Ports par défaut	API 8000 ; P2P 9000

Annexe B. Interfaces principales

Route	Rôle
GET /info, GET /state	État du réseau : hauteur, difficulté, récompense, supply.
GET /chain?from_height=N&limit=100	Blocs par lots de 100 au plus.
GET /mempool	Transactions en attente.
GET /balance/{adresse}, GET /utxos/{adresse}	Solde et sorties non dépensées.
GET /miners	Statistiques et classement des mineurs.

Route	Rôle
GET /mining/tip	Pointe de la chaîne.
GET /mining/template?miner_pub=...	Modèle de bloc à miner.
POST /mining/submit	Soumission d'un bloc miné.
POST /transaction/receive	Transaction signée par son propriétaire.
POST /block/receive	Bloc reçu d'un pair ou de la synchronisation.

Annexe C. Ressources

- Site : <https://cryptomonnaiesia.com>
- Nœud : <https://github.com/julienfiquet/cmia-node>
- Mineur : <https://github.com/julienfiquet/cmia-miner>